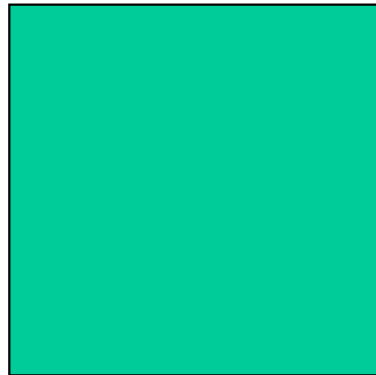


Modelado II: Texturizado



Temario

Tema 1: Introducción: Revisión histórica da computación gráfica

Tema 2: Estándares gráficos Evolución histórica, OpenGL vs DirectX

Tema 3: Formas 2D e Antialiasing (final del curso)

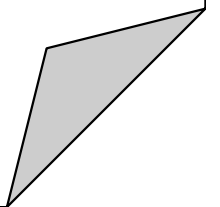
Tema 4: Transformacións xeométricas, 2D, 3D,

Tema 5: Proyeccións, modelo de cámara sintética.

Tema 6: Modelado e texturas 50%

Tema 7: Color, Iluminación y sombreado

Tema 8: Determinación de superficies visibles, z-Buffer



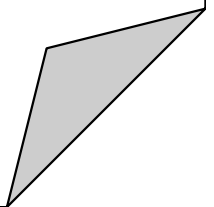
Temario: Tema 6: Modelado y texturas

Tema 6: Modelado y texturas.

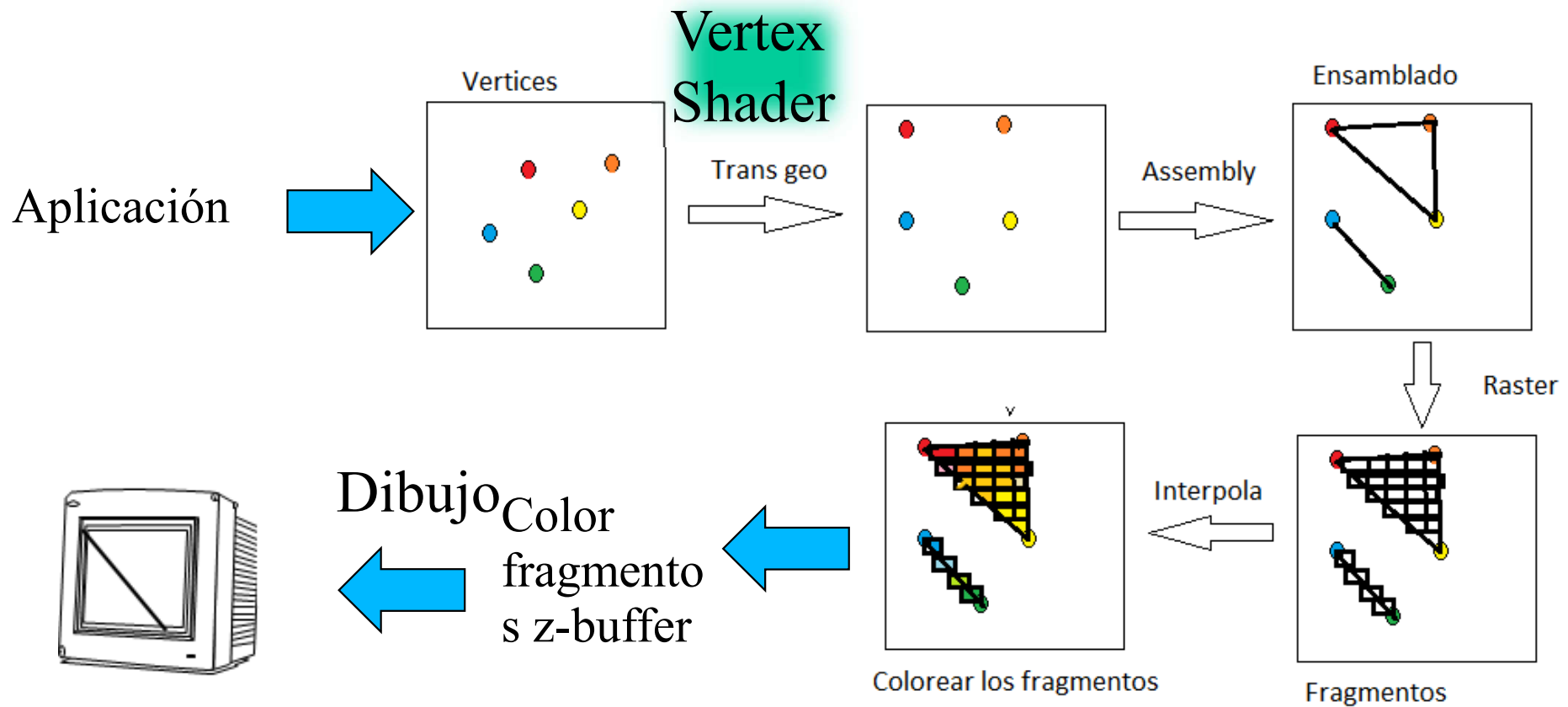
6.1. Diseño avanzado de objetos

6.2. Librerías auxiliares

6.3. Texturización.

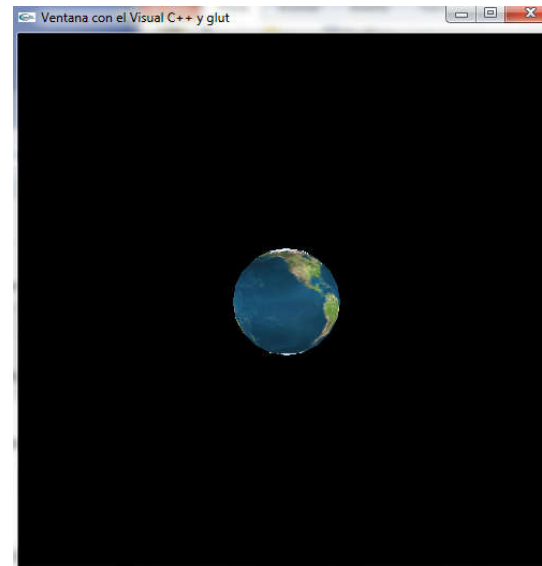


Pipe



Introducción

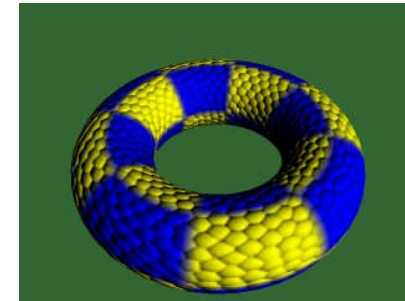
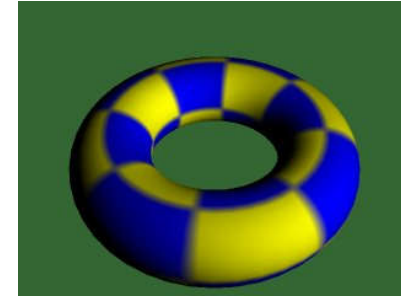
- La utilización de texturas:
 - Aumentan el realismo de la imagen sin aumentar el número de polígonos.
 - Proporciona una clave para la percepción de la profundidad.
 - Permite simular multitud de efectos FX: sombras, reflejos, entornos.
 - Problema Ocupan espacio...
- Texturas: colección o array n-dimensional de valores que se representan sobre la geometría.



Texturizado

El texturizado consiste en superponer sobre los polígonos imágenes que aumenten su realismo. Existen varias técnicas de texturizado:

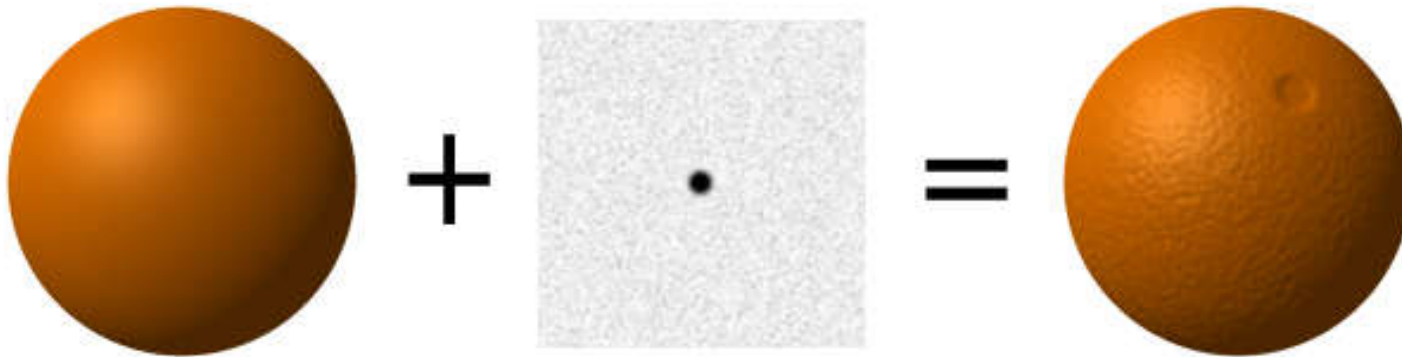
- **Texture Mapping**: Crea una textura a partir de un patrón (imagen) sobre la superficie.
- **Bump Map**: Perturba la superficie mediante una modificación de las normales (puede no considerado mapeado en su sentido estricto).
- **Environmental Map**: Mapea el entorno sobre la superficie. (e.g. reflections, light mapping).
- **Light mapping**. Mapea un mapa de iluminación sobre el objeto.



DIFFUSE x LIGHTMAP

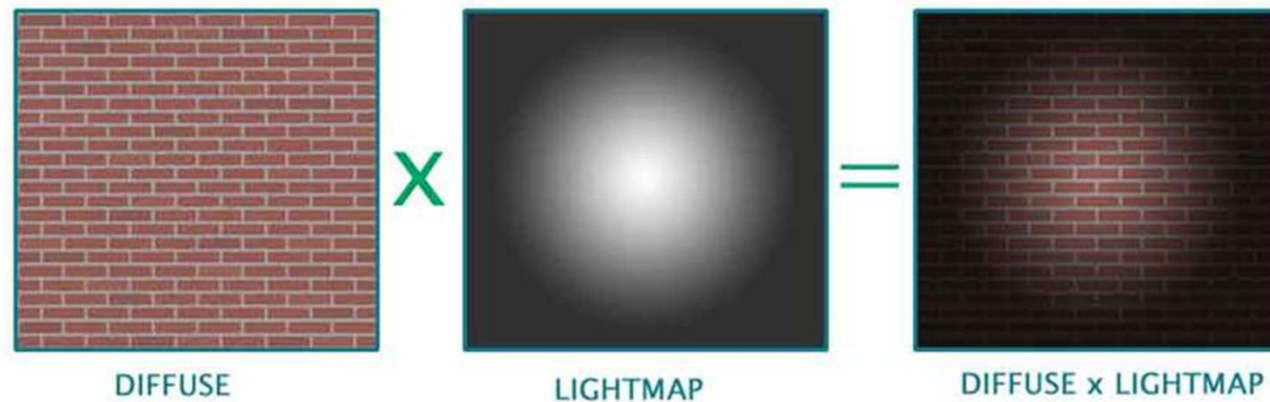
Texturas: Bump mapping

- Mapeado topológico 1978 James Blind.
- Modifica las normales de las caras de un objeto sin cambiar su geometría.
 - Se utiliza un mapa de alturas, una matriz de datos $2^n * 2^n$ generalmente una imagen.
 - Se calcula la normal en función de ese mapa de alturas.
 - Se combina las normales con la geometría del objeto.
 - Se calcula el modelo de iluminación para esos nuevos valores de normales.



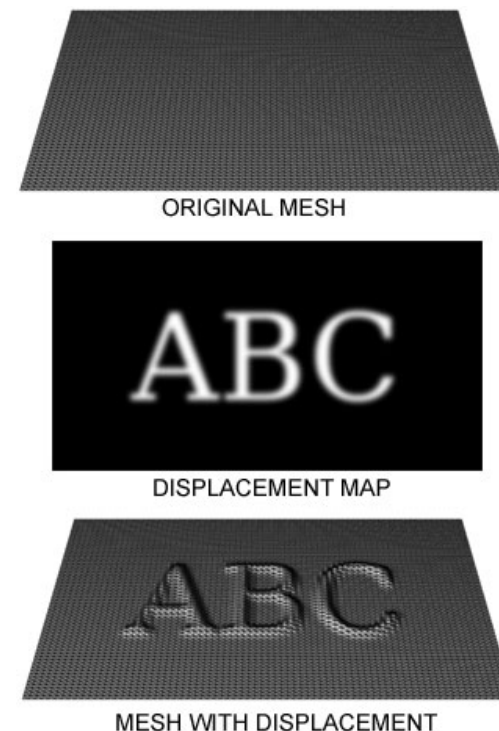
Light mapping.

- El brillo de las superficies en una escena se calcula previamente y se almacena en mapas de textura para su uso posterior. Los mapas de luz se aplican con mayor frecuencia a objetos estáticos para proporcionar efectos de iluminación como la iluminación global a un costo muy bajo.

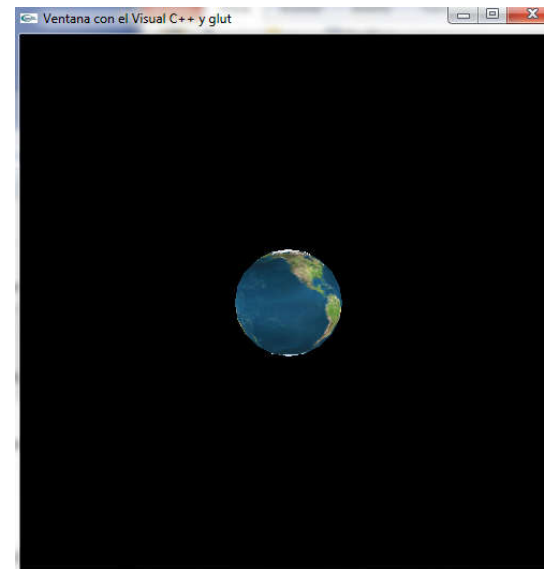
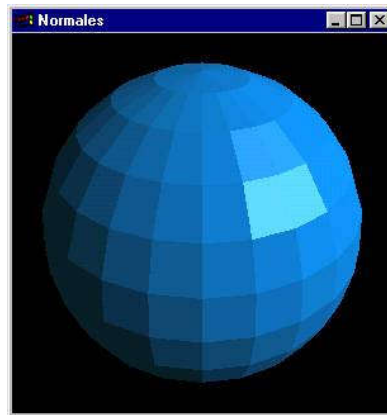


Texturas: Displacement mapping

- Modifica la geometría de un objeto mediante un mapa de texturas.
- En general en la dirección de las normales de la misma.
- Genera una gran cantidad de geometría adicional
- Ventajas frente al bump mapping.
 - Puede genera sombras.
 - Pueden genera oclusiones.
 - Puede usar para sombreados.
 - Desventajas es lento.
 - Geometry shader.

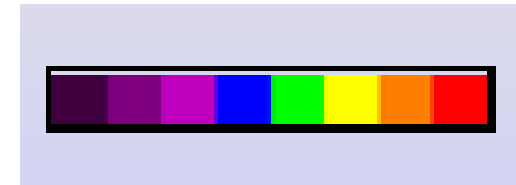


Texture Mapping



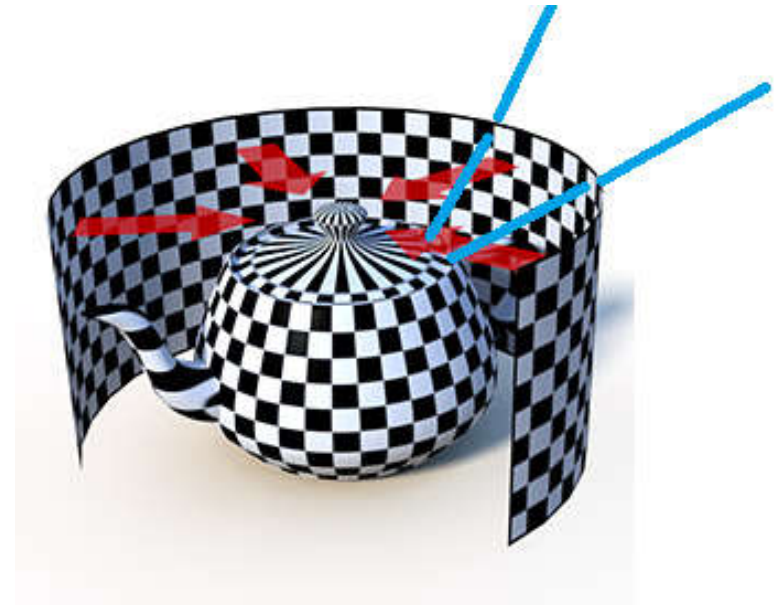
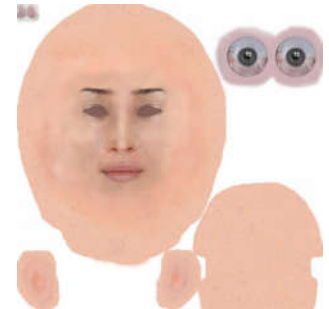
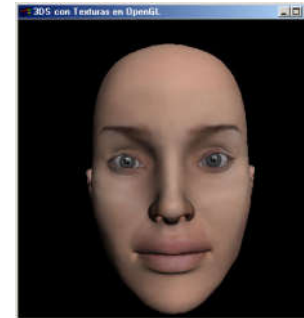
Mapeado, tipo texturas.

- Es posible utilizar tres tipos.
 - Texturas 1D.
 - Son imágenes sin alto, sólo con ancho.
 - **Texturas 2D.**
 - **Son imágenes Bitmaps ordinarias.**
 - Texturas 3D.
 - Sólo las aceptan algunos sistemas.
Ocupan mucha memoria.
- Texturas procedurales.



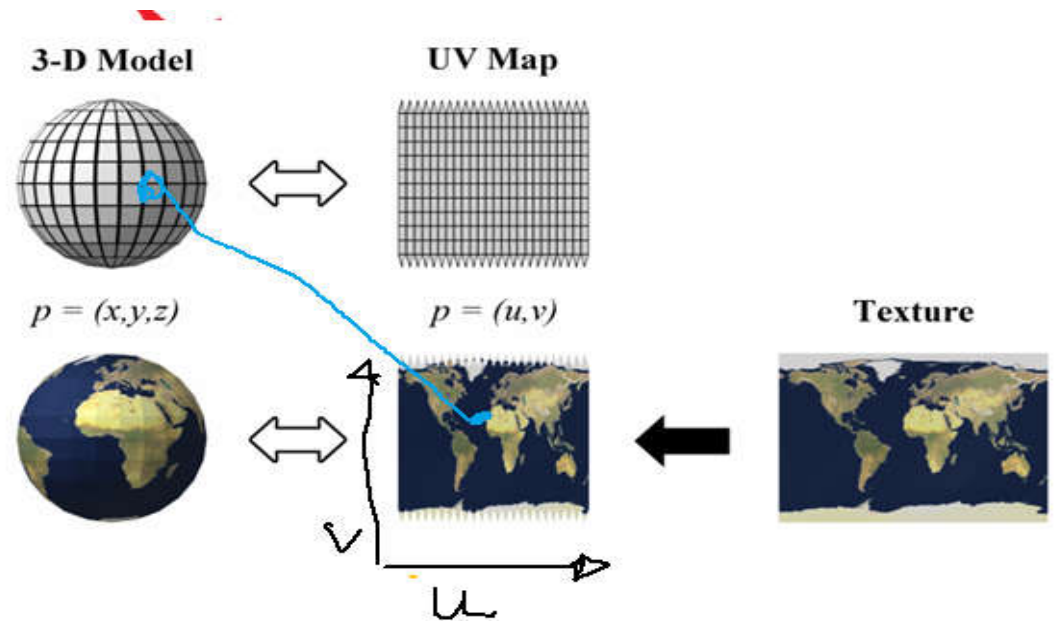
Como se aplican las texturas; Mapeado

- Directo
 - La textura se mapea sobre el objeto
- Dos fases.
 - Se mapea la imagen sobre un objeto 3D.
 - Esférico
 - Cilíndrico.
 - Otros
 - Se proyecta sobre el objeto.



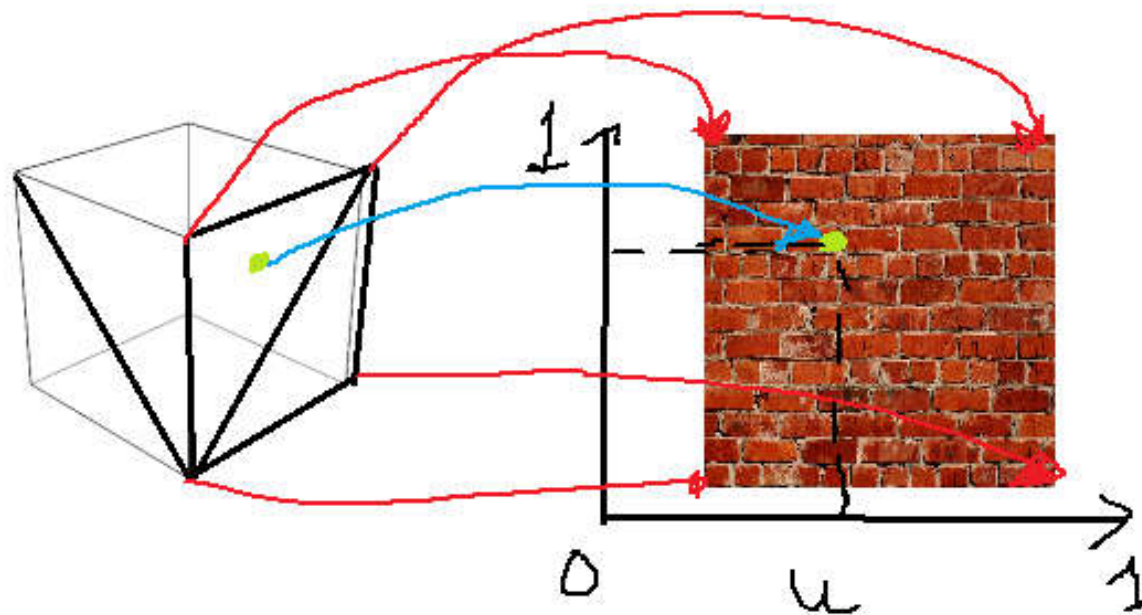
Mapeado directo de texturas (u, v).

- Para calcular el valor de un pixel determinado es necesario utilizar la relación entre la textura (u, v) y los vértices superficie.
- Objetivo es obtener las coordenadas de textura; unos valores que nos permita indicar que coordenada de la textura corresponde a cada vértice.
- En general este proceso es muy complejo y suele utilizar un programa de diseño CAD para obtener las coordenadas de texturas.



Coordenadas U,V

- Las **coordenadas U,V** son un sistema de coordenadas utilizado para mapear texturas en superficies 3D. Estas coordenadas definen cómo una imagen 2D (la textura) se "envuelve" o se proyecta sobre un objeto 3D. Son análogas a las coordenadas X e Y en un plano 2D, pero se usan específicamente para texturas.



Mappeado de Texturas (OpenGL)

- Habilitar. 3D, 2D or 1D.
- Nombrar la textura. (**índice**).

- Almacenamiento en memoria.
 1. Dar valores o leer un fichero.
- Definir la textura dar propiedades.
 - Describir el tipo de imagen.
 - Qué hacer ante imágenes demasiado pequeñas o demasiado grandes.

- Pegar textura (u, v).
 - Describir como colocar la imagen sobre el objeto.
- Activar la textura (hacerla operativa) para cada objeto.

3.3.

```
glBindTexture(GL_TEXTURE_2D, texture);
```

```
layout (location = 0) in vec3 aPos;  
layout (location = 1) in vec3 aColor;  
layout (location = 2) in vec2 aTexCoord;  
out vec2 TexCoord;  
out vec3 ourColor;  
uniform mat4 transform;  
  
void main() {  
    gl_Position = transform * vec4(aPos, 1.0);  
    ourColor = aColor;  
    TexCoord = vec2(aTexCoord.x, aTexCoord.y);}
```

```
#version 330 core  
in vec3 ourColor;  
in vec2 TexCoord;  
uniform sampler2D texture;  
  
void main(){  
    gl_FragColor = texture(texture, TexCoord);  
}
```


Habilitar el mapeado de texturas

En el main() o en “InitOpengl()”:

```
glEnable(GL_TEXTURE_2D);
```

Si quieres que la superficie aparezca sin textura con el color original será necesario deshabilitarlo.

```
glDisable(GL_TEXTURE_2D);
```

Se activar o desactivar.

```
glBindTexture(GL_TEXTURE_2D, Indice);  
glBindTexture(GL_TEXTURE_2D, 0);
```

Mappeado de Texturas (OpenGL)

- Habilitar. 2D or 1D
- Nombrar la textura (índice) 
- Almacenamiento en memoria.
 - Dar valores.
 - Leer un fichero.
- Definir la textura.
 - Describir el tipo de imagen.
 - Que hacer ante imágenes demasiado pequeñas o demasiado grandes.
- Pegar textura.
 - Describir como colocar la imagen sobre el objeto.
- Activar la textura (hacerla operativa).

Leer la y Activar la textura (hacerla operativa).

```
void glGenTextures( GLsizei n, GLuint * textures );
```

- *n* Es el número de índices a crear.
- *textures* Un puntero al primer elemento del array en donde almacenarán los índices creados

```
int textura;  
glGenTextures(1, &textura);  
int arraytex[];  
glGenTextures(n, &arraytex[]);
```

```
void glBindTexture( GLenum target, GLuint texture );
```

- *target* : GL_TEXTURE_1D o GL_TEXTURE_2D.
- *texture*: índice de la textura que queremos enlazar (no en uso)

```
- glBindTexture(GL_TEXTURE_2D, texture[0]);
```

```
- Esto sirve tanto para hacerla operativa como usarla después al vuelo como las listas o los VAO o VBO.
```

```
- glBindTexture(GL_TEXTURE_2D, 0); // no queremos usar ninguna
```

```
- glDeleteTextures (GLsizei num, const GLuint *nombres)
```

Mappeado de Texturas (OpenGL)

- Habilitar. 2D or 1D
- Activar la textura.
- Almacenamiento en memoria. 
 - Dar valores.
 - Leer un fichero.
- Definir la textura.
 - Describir el tipo de imagen.
 - Como realizar el mapeado y el rasterizado
- Pegar textura.
 - Describir como colocar la imagen sobre el objeto.
- Activar la textura (hacerla operativa)

Almacenar en memoria. (256*256)

Una textura es especificada por un array de **texels**, podríamos crearlo manualmente :

GLubyte imageData1[3 * 256 * 256]

Un byte para cada color RGB. ancho Alto

```
unsigned char Colores[4][4][3] = {  
    {{64,0,0}, {128,0,0}, {192,0,0}, {255,0,0}}, // Rojos  
    {{0,64,0}, {0,128,0}, {0,192,0}, {0,255,0}}, // Verdes  
    {{0,0,64}, {0,0,128}, {0,0,192}, {0,0,255}}, // Azules  
    {{64,64,0}, {128,128,0}, {192,192,0}, {255,255,0}} // Amarillos  
};
```

Para crear una textura:

- 1) Escribir un programa que la genere, procedurales.
- 2) Leerla de un fichero, imágenes.

Leerla de un fichero.

- Usamos librerías auxiliares:
- `//Librerías de cabecera para manejo de imagenes.`

```
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>
```

```
int myCargaTexturas(char *nome){
int textura;
glGenTextures(1, &textura);
glBindTexture(GL_TEXTURE_2D, textura);
int width, height, nrChannels;                                //RBG, RGBA
unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);

.....
}
unsigned char *stbi_load(
char const *filename, // Ruta del archivo de imagen
int *x,               // Ancho de la imagen (load)
int *y,               // Alto de la imagen (load)
int *channels_in_file, // Canales de color en la imagen (load)
int desired_channels  // Canales deseados en la salida
```

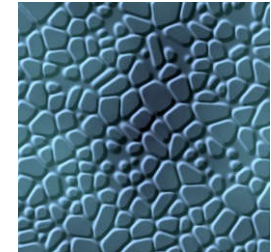
Desired_channels:

Número de **canales deseados** en la salida.

- 0: Usar los canales originales de la imagen.
- 1: Forzar la imagen a escala de grises.
- 3: Forzar la imagen a RGB.
- 4: Forzar la imagen a RGBA.

Texturas procedurales

- La “imagen” es calculada a partir de una función matemática.
- Se precisa un algoritmo una función R^n
- Ocupa poco espacio.
- Se puede re-escalar fácilmente.
- Útil para superficies grandes o pequeñas
- No consume memoria sino CPU.
- Tipos de mapas (ejemplo)
 - Geométricos.
 - Patrón repetitivo.
 - Ruido fractal.
 - Funciones de turbulencia.
- Generación de mapas de juegos y fenómenos naturales.



Mappeado de Texturas (OpenGL)

- Habilitar. 2D or 1D
- Activar la textura.
- Almacenamiento en memoria.
 - Dar valores.
 - Leer un fichero.
- Definir la textura.
 - Describir el tipo de imagen.
 - Como realizar el mapeado y el rasterizado
- Pegar textura.
 - Describir como colocar la imagen sobre el objeto.
-



Texturas: De array de datos a textura.

- `void glTexImage2D(GLenum objetivo, GLint nivel, GLint componentes, GLsizei ancho, GLsizei alto, GLint borde, GLenum formato, GLenum tipo, const GLvoid *pixels);`

- *objetivo:* define que textura hay que definir y debe ser `GL_TEXTURE_2D` (1D).
- *nivel:* Indica el nivel de detalle de las texturas y es normalmente 0.
- *Componentes:* Indica el número de componentes de color usados en cada píxel.
- *Ancho:* Especifica el ancho de la textura. Debe ser potencia de 2.
- *Alto:* Especifica la altura de la textura. Debe ser potencia de 2.
- *Borde:* Controla el número de píxels de borde que el OpenGL utilizará
- *Formato:* Indica el tipo de valores de color esperados.
- *Tipo:* Tipo de la variable que almacena los valores de color por píxel.
- *Pixels :* Puntero a la variable que almacena los colores por píxel.

```
glTexImage2D(GL_TEXTURE_2D, 0, 3, width2, height2, 0, GL_RGB,  
             GL_UNSIGNED_BYTE, imageData2);
```

Ejemplo.

```
// Es necesario para que el stb_image funcione
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>

int myCargaTexturas(char *nome){
    int textura;
    glGenTextures(1, &textura);
    glBindTexture(GL_TEXTURE_2D, textura);
    // load image, create texture and generate mipmaps
    int width, height, nrChannels;
    unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
    //Parametros
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    stbi_image_free(data);
    return (textura);
}
```

Ejemplo // Load Bitmaps And Convert To Textures

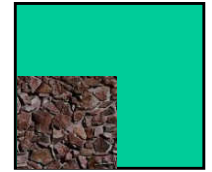
```
// Es necesario para que el stb_image funcione
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>

int myCargaTexturas(char *nome){
    int textura;
    glGenTextures(1, &textura);
    glBindTexture(GL_TEXTURE_2D, textura);
    // load image, create texture and generate mipmaps
    int width, height, nrChannels;
    unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
    //Parametros
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
    stbi_image_free(data);
    return (textura);

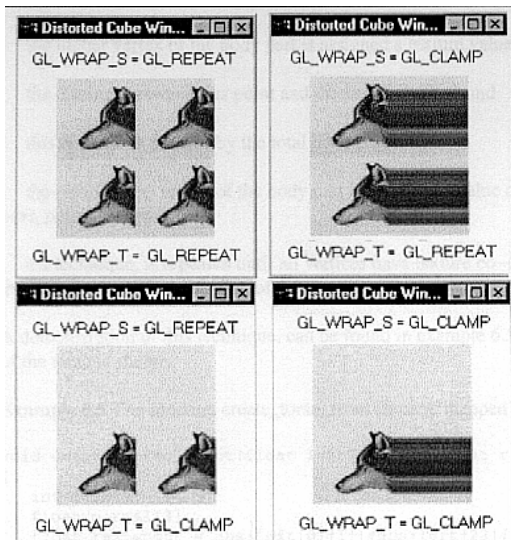
    • }
```

Problemas de tamaño: glTexParameterf



Deben ser llamadas cuando en un array de datos `glTexImage2D()` es activo y uno hay un `glBindTexture(GL_TEXTURE_2D, 0);`

- a) Especifica que ocurre si la textura no cubre a los límites del objeto Warp o warping process. Así especifica si es repetida *repeat* o estirada *clamp*. Por tanto, la función debe ser llamada 2 veces.



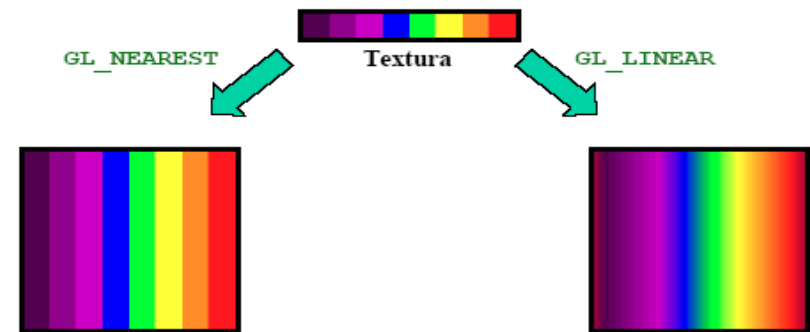
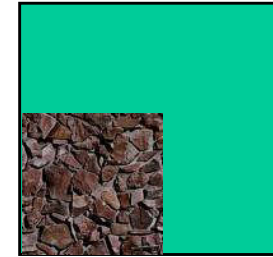
```
glTexParameterf( GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,  
GL_CLAMP);
```

```
glTexParameterf( GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,  
GL_REPEAT);
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

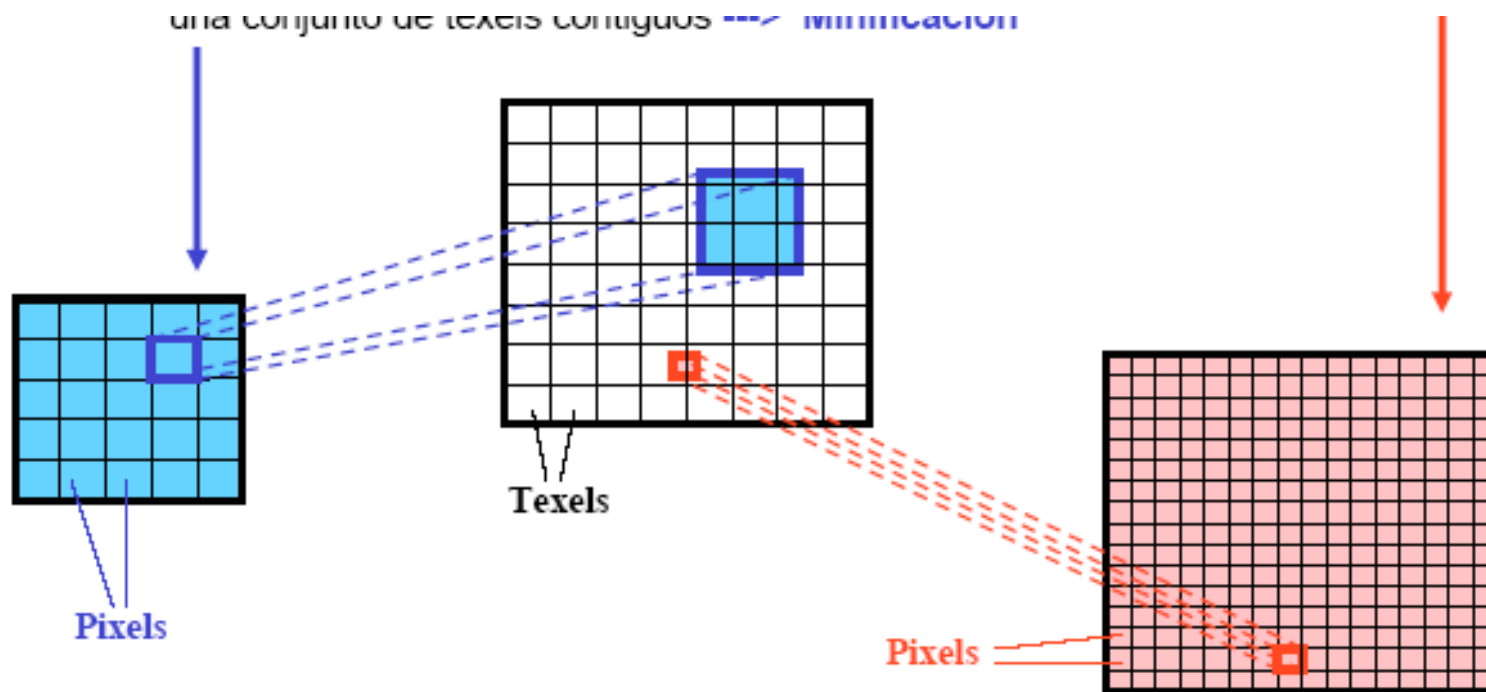
Parámetros de Texturas

- Sobre el Objeto: Problemas de tamaño Que hacer si la textura no abarca todo el objeto.
- Sobre la pantalla: Problemas de escala: Que hacer al rasterizar la textura es más grande o más pequeña que su tamaño original. Ocupa un más de un pixel o menos sobre la pantalla.
- Otros efectos Mezclas: Que hacer si el objeto final tenía un color.

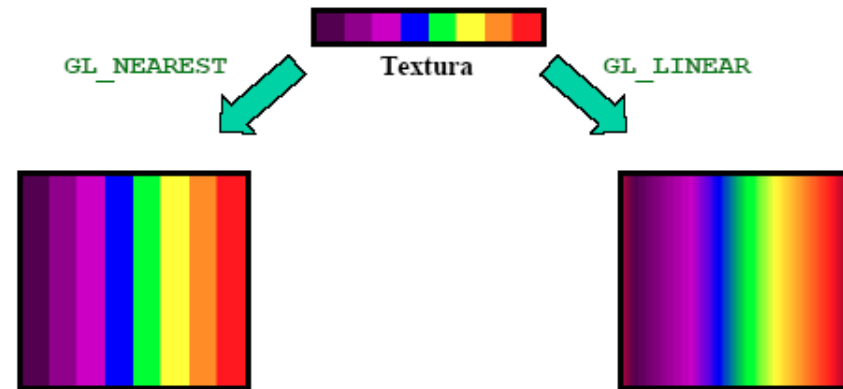


Problemas de escala: *glTexParameterf*

- ¿Qué pasa si cada pixel de la imagen no se corresponde en tamaño a lo que se ve en pantalla.
 - Al mapear la imagen de textura sobre una superficie, los texels no se van a corresponder exactamente con los pixels
 - Si la superficie es mayor que la textura, cada pixel se corresponderá con un trozo pequeño de texel ---> **Magnificación**
 - Si la superficie es menor que la textura, cada pixel se corresponderá con un conjunto de texels contiguos ---> **Minificación** Texels



Problemas de escala: *glTexParameterf*



GL_NEAREST



GL_LINEAR

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

Mappeado de Texturas (OpenGL)

- Habilitar. 2D or 1D
- Almacenamiento en memoria.
 - Dar valores.
 - Leer un fichero.
- Nombrar la textura.
- Definir la textura.
 - Describir el tipo de imagen.
 - Que hacer ante imágenes demasiado pequeñas o demasiado grandes.
- Pegar textura. 
 - Describir como colocar la imagen sobre el objeto.

Con OpenGL 3.3

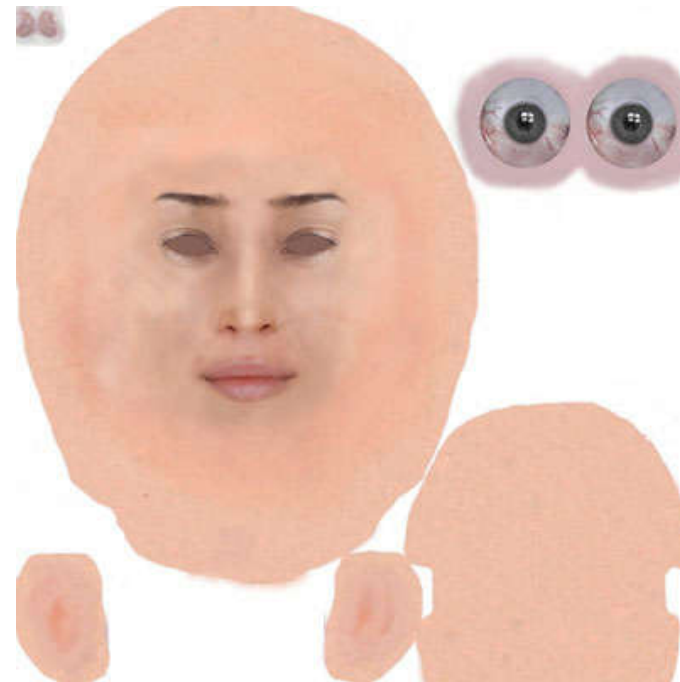
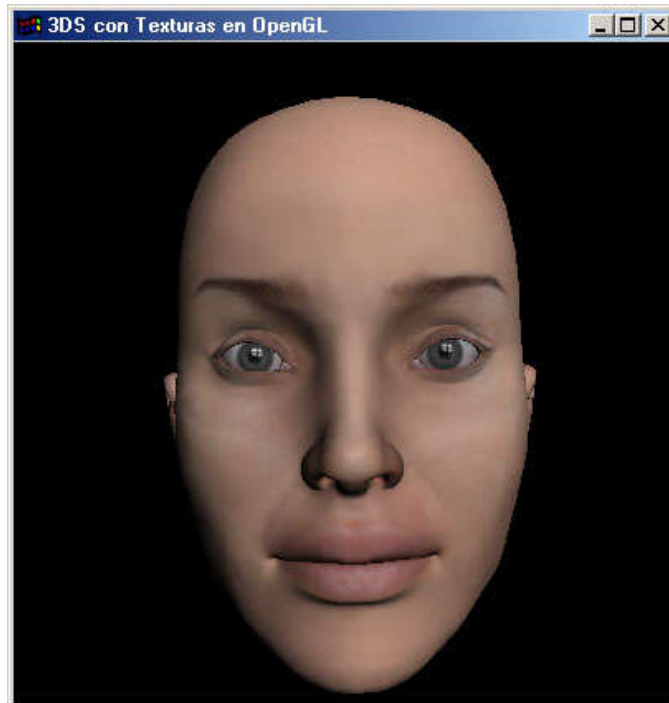
Especificar las coordenadas de textura.

```
float vertices[] = {  
    //Frontal vertices           Normales           texturas  
    -0.5f, -0.5f,  0.5f,        0.0f, 0.0f, 1.0f,        0.0, 0.0,  
    0.5f, -0.5f,  0.5f,        0.0f, 0.0f, 1.0f,        1.0, 0.0,  
    0.5f,  0.5f,  0.5f,        0.0f, 0.0f, 1.0f,        1.0, 1.0,  
    0.5f,  0.5f,  0.5f,        0.0f, 0.0f, 1.0f,        1.0, 1.0,  
    -0.5f,  0.5f,  0.5f,        0.0f, 0.0f, 1.0f,        0.1, 1.0,  
    -0.5f, -0.5f,  0.5f,        0.0f, 0.0f, 1.0f,        0.0, 0.0,  
    .....  
    // vertices  
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);  
    glEnableVertexAttribArray(0);  
    // Normales  
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float)));  
    glEnableVertexAttribArray(1);  
    // texture  
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(6 * sizeof(float)));  
    glEnableVertexAttribArray(2);
```

Programas de diseño

Los programas de diseño ya nos proporcionan las coordenadas de textura (u,v).

También se pueden calcular como en sphere, que os pasé.



Múltiples texturas

Vemos los pasos a seguir.

1) Leer dos texturas y asignarles distinto identificador, textura, textura 2.

2) Modificar el shader.

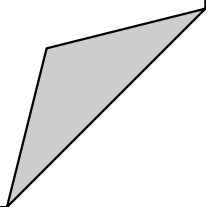
```
in vec2 TexCoord;  
uniform sampler2D texture1;  
uniform sampler2D texture2;  
void main(){  
    FragColor = mix(texture(texture1, TexCoord), texture(texture2, TexCoord), 0.3);  
}
```

3) Mapear las texturas sobre las variables uniformes.

```
glUseProgram(shaderProgram);  
glUniform1i(glGetUniformLocation(shaderProgram, "texture1"), 0);  
glUniform1i(glGetUniformLocation(shaderProgram, "texture2"), 1);
```

4) Usarlas

```
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, texture);  
glActiveTexture(GL_TEXTURE1);  
glBindTexture(GL_TEXTURE_2D, texture2);
```



3.3.

```
glBindTexture(GL_TEXTURE_2D, texture);
```

```
layout (location = 0) in vec3 aPos;  
layout (location = 1) in vec3 aColor;  
layout (location = 2) in vec2 aTexCoord;  
out vec2 TexCoord;  
out vec3 ourColor;  
uniform mat4 transform;  
  
void main() {  
    gl_Position = transform * vec4(aPos, 1.0);  
    ourColor = aColor;  
    TexCoord = vec2(aTexCoord.x, aTexCoord.y);}
```

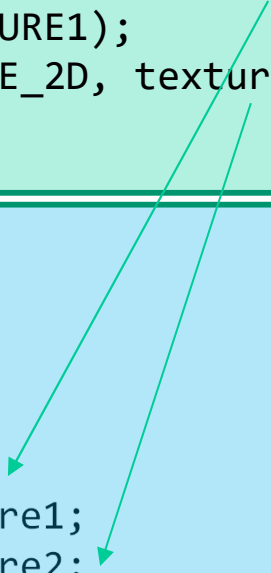
```
#version 330 core  
in vec3 ourColor;  
in vec2 TexCoord;  
uniform sampler2D texture;  
  
void main(){  
    gl_FragColor = texture(texture, TexCoord);  
}
```

Mapeado multicapa 3.3

En OpenGL 3.3 es posible y fácil utilizar más de una textura a la vez sobre un objeto. Para esto se utilizan las denominadas Unidades de texturas que permiten ir activando o desactivando las texturas manualmente.

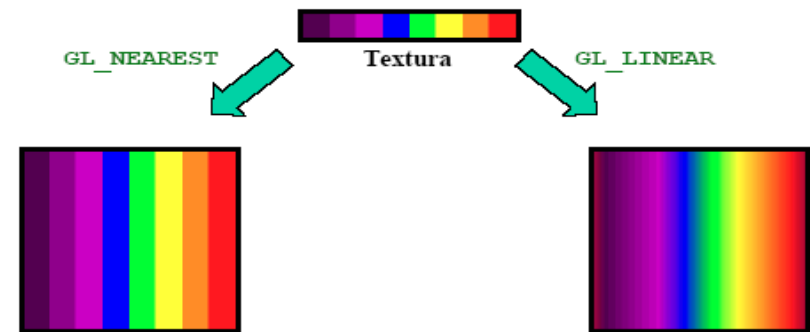
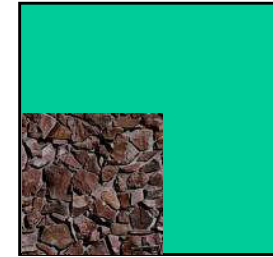
```
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, texture1);  
glActiveTexture(GL_TEXTURE1);  
glBindTexture(GL_TEXTURE_2D, texture2);  
sphere.draw();
```

```
#version 330 core  
out vec4 FragColor;  
in vec3 ourColor;  
in vec2 TexCoord;  
// texture sampler  
uniform sampler2D texture1;  
uniform sampler2D texture2;  
  
void main() {  
    FragColor = mix(texture(texture1, TexCoord), texture(texture2, TexCoord), 0.9);  
}
```



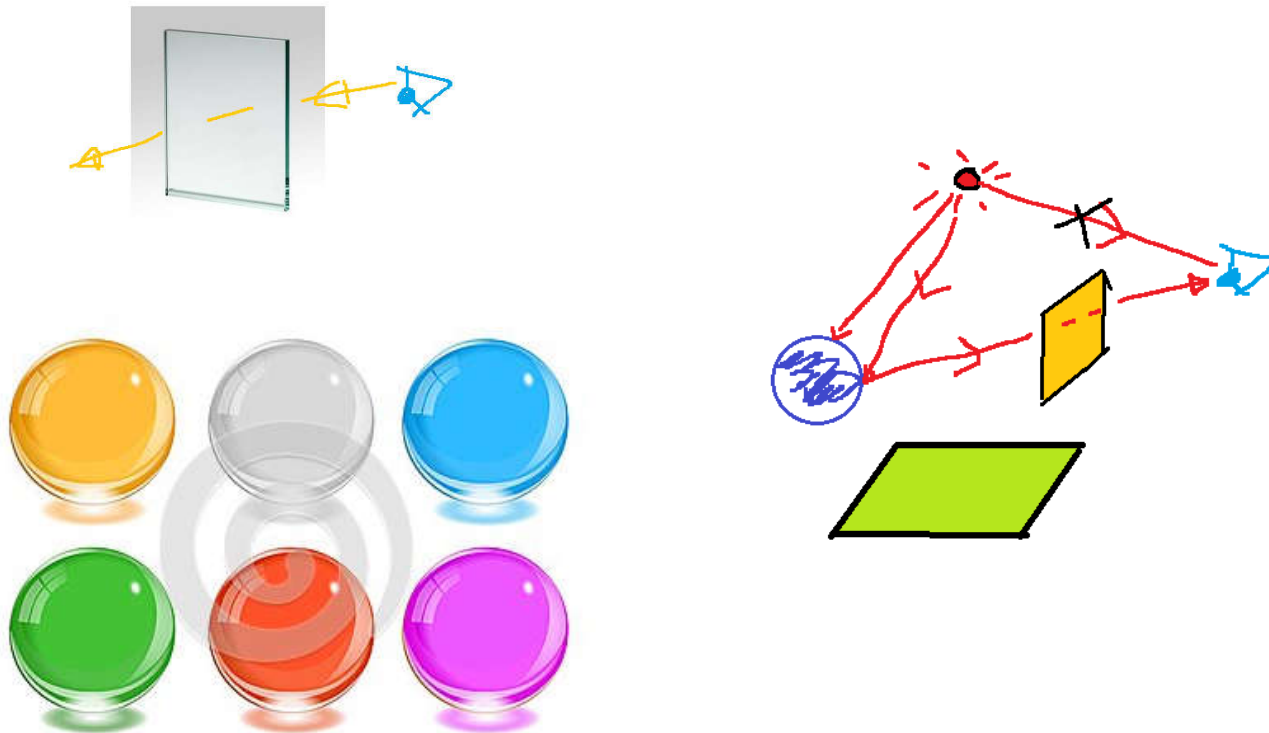
Parámetros de Texturas

- Sobre el Objeto: Problemas de tamaño Que hacer si la textura no abarca todo el objeto.
- Sobre la pantalla: Problemas de escala: Que hacer al rasterizar la textura es más grande o más pequeña que su tamaño original. Ocupa un más de un pixel o menos sobre la pantalla.
- Otros efectos Mezclas: Que hacer si el objeto final tenía un color.



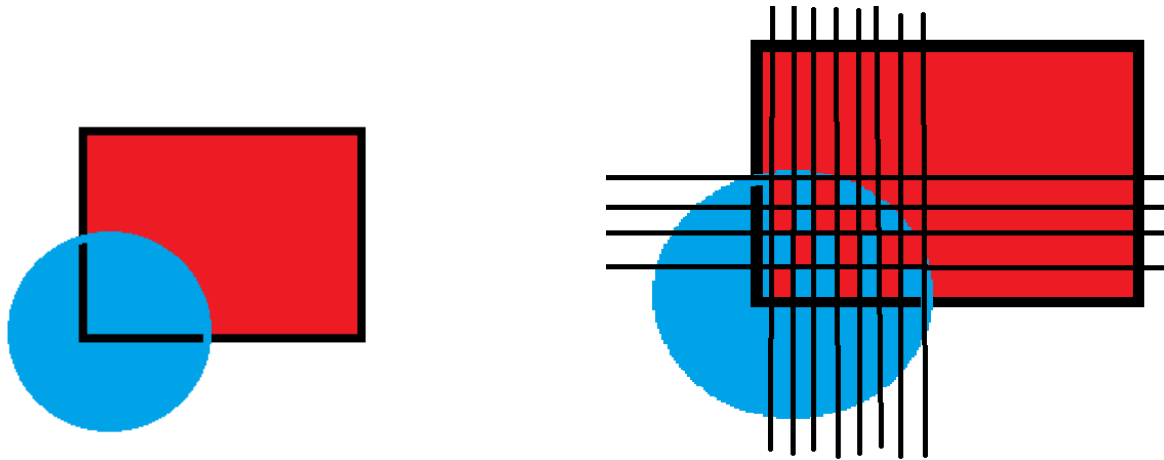
Transparencia Alpha

Opengl puede simular los objetos transparentes.



Blending-Mezcla

Forma más sencilla de hacer una transparencia, es mediante la técnica de la puerta o ventana transparente.

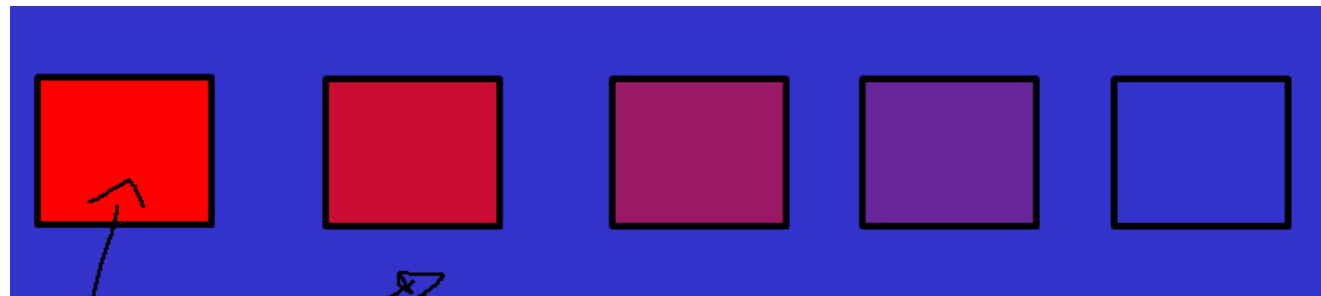


Alpha

Forma más sencilla de hacer una transparencia, es mediante la técnica de la puerta o ventana transparente.

Alpha = 1 opaco.

Alpha = 0 transparente



Source

Target

$$c = \alpha_s c_s + (1 - \alpha_s) c_t$$

Se utiliza una ecuación de la forma.

Como se usa.

Canal alfa en colores RGBA (0.0 = totalmente transparente, 1.0 = opaco).

Habilitar blending:

```
glEnable(GL_BLEND);
```

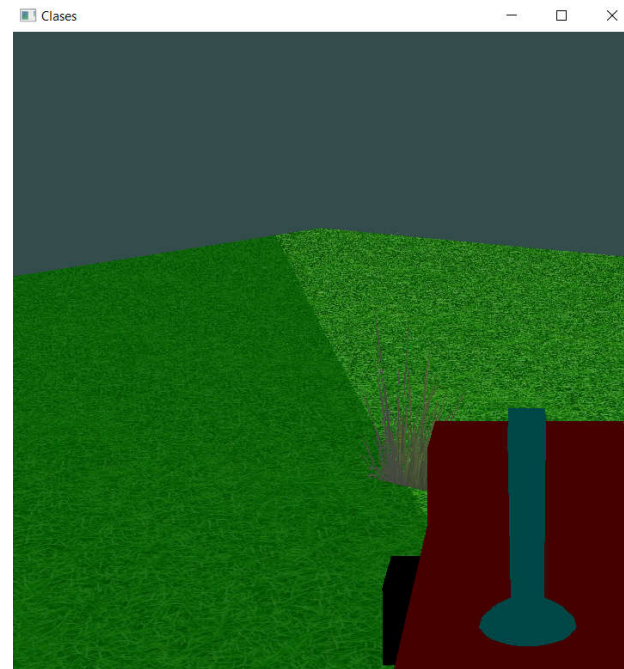
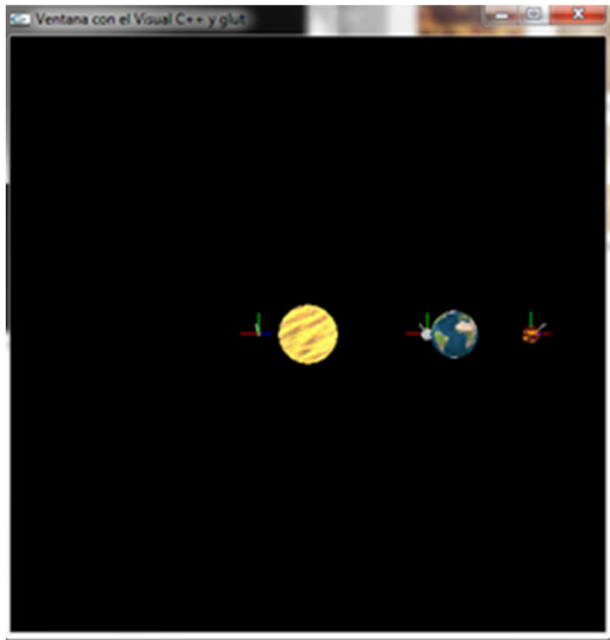
```
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

- GL_SRC_ALPHA: usa el alfa del fragmento actual.
- GL_ONE_MINUS_SRC_ALPHA: calcula la contribución del fondo.

•Orden de dibujo:

- Primero dibujar objetos opacos.
- Luego los transparentes **de atrás hacia adelante** (painter's algorithm).

Prácticas



Temario

Tema 1: Introducción.

Introducción *100%*

Revisión histórica da computación gráfica. *100%*.

Tema 2: Estándares gráficos Evolución histórica, OpenGL *100%*

Tema 3: Formas 2D e Antialiasing (final del curso) *50%*

Tema 4: Transformacións xeométricas, 2D, 3D. *100 %*

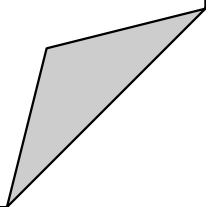
Tema 5: Proyeccións, modelo de cámara sintética. *100%*

Tema 6: Modelado e texturas *100%*

Tema 7: Color, Iluminación y sombreado *100%*

Tema 8: Determinación de superficies visibles, z-Buffer *50%*

Tema 9: Shaders GLSL 0%



Bibliografía

- GRÁFICOS POR COMPUTADORA CON OPENGL, Hearn, Donald ; Baker, Pauline.
 - Capítulo 10 pp 650-658, pp 668-673.
- Opengl Reedbook version 2. Opengl 1.1. Capitulo 9.